- OBBIETTIVO

In base alle nozioni apprese in classe questa settimana, quest'oggi dovremo andare ad individuare:
dapprima leggendo il codice, per poi trascrivendo questo report, le anomalie nel codice ricevuto. Mettendo in pratica le nozioni apprese,potremo facilmente individuare gli errori.

NOZIONI APPRESE:

Gli errori possono essere di causa sintattica e di causa logica:

- Errori di Sintassi (SyntaxError): Rilevati dall'interprete prima dell'esecuzione, indicano una struttura errata del codice (es. due punti mancanti, parentesi aperte e non chiuse).
- Errori Semantici (Logici): Il programma gira senza crashare, ma produce risultati errati a causa di una logica algoritmica errata.
- Errori di Runtime: Si verificano durante l'esecuzione del programma anche se la sintassi è corretta.

1. *NameError: Uso di una variabile non definita.*
2. *TypeError: Operazione applicata a un tipo di dato inappropriato.*
3. *ValueError: Una funzione riceve un argomento del tipo corretto ma con valore inappropriato.*
4. *IndexError: Indice di una sequenza fuori intervallo.*
5. *KeyError: Chiave non trovata in un dizionario.*
6. *ZeroDivisionError: Divisione per zero*

- ***LEGGERE IL CODICE RIGA PER RIGA, PER CERCARE DI INTERPRETARE LE SUE VOLONTA'***

Riga 1 — Viene importato il modulo datetime della libreria standard Python, necessario per ottenere la data odierna e l'ora attuale.

Riga 3 — Inizia un ciclo while True, un ciclo infinito che continua finche non viene interrotto con un break. L'intenzione e tenere il programma in attesa continua dei comandi dell'utente.

Riga 4 — La funzione input() mostra il prompt 'Cosa vuoi sapere?' e attende che l'utente digiti qualcosa. Il testo viene salvato nella variabile comando_utente.

Righe 5-9 — Struttura condizionale con due casi: se l'utente ha scritto 'esci' (riga 5), il programma stampa 'Arrivederci!' (riga 6) e dovrebbe uscire con il break; negli altri casi (else riga 8), dovrebbe passare il comando alla funzione assistente_virtuale() e stamparne il risultato (riga 9). Risultato non veritiero. Errore che compromette totalmente il funzionamento del codice ##

Riga 11 — Viene definita la funzione assistente_virtuale(comando), che riceve la stringa digitata dall'utente e restituisce la risposta appropriata. La sua definizione appare dopo il ciclo che la usa: errore di ordine di esecuzione analizzato piu avanti.

Righe 12-21 — La funzione gestisce quattro casi tramite if/elif/else: risponde alla domanda sulla data (righe 12-14),sull'ora (righe 15-17), sul nome (righe 18-19), e per qualsiasi altro input restituisce 'Non ho capito la tua domanda.' (righe 20-21). In tutti i casi costruisce una variabile risposta che restituisce alla riga 22.

Riga 22 — La funzione restituisce risposta al chiamante, che la stampera a schermo tramite la riga 9.##

Possiamo facilmente intuire che si cerchi di far interagire un assistente virtuale , legge i comandi dell'utente, li passa a una funzione dedicata e termina su 'esci'

ANALISI ERRORI

1. ***Il primo errore riscontrato e' la mancanza dei : nella 3^ riga, dove c'e' l'inizio del ciclo wihile true***

2. ***Il secondo errore lo troviamo nella posizione del while True, e' in errore perche si trova prima del blocco di codice della funzione***

3. ***Il terzo errore che ho riscontrato si trova nella 13^ riga***
   Oggi = datetime.datetoday()
   L'errore sta nell scrittura di datetoday prima dopo il . La formula giusta dovrebbe essere: oggi = datetime.date.today()

4. ***Il quarto invece lo abbiamo riscontarto nella riga 16^ dove nella scrittura dei dati la parola TODAY non andava scritta.***
   Forma giusta datetime.datetime.now()

5. ***Il quinto errore ci porta a tornare indietro nel codice allaLe righe 5-9 contengono l'unico vero errore logico del programma, ma e un errore che si ramifica in piu conseguenze visibili. Il punto di partenza e la riga 7: il break e scritto con lo stesso livello di indentazione del blocco if anziche trovarsi al suo interno. In Python l'indentazione non e estetica — definisce la gerarchia del codice. Come scritto, il break appartiene al ciclo while direttamente, non all'if: viene quindi eseguito sempre, a ogni iterazione, qualunque cosa l'utente abbia digitato Da questo errore derivano due conseguenze immediate. La prima: il ciclo termina sempre al primo giro, rendendo il programma completamente inutilizzabile l'assistente chiede l'input e si chiude senza niente da elaborare.***
   La seconda: il print ('Arrivederci!') alla riga 6 si trova dentro l'if, ma il break che dovrebbe seguirlo e fuori dalle istruzioni che formano un'unica azione logica sono strutturalmente disallineate. Infine, l'else alla riga 8 rimane da solo, e il suo if di riferimento viene interrotto dal break, rendendo il blocco else irraggiungibile.

- ## RIMEDI NELLA SCRITTURA

Riga 3 Aggiungere ':' subito dopo True,
ottenendo: while True: Un solo
carattere sblocca l'intero script.

Riga dalla 5 alla 8 Portare il break dentro il blocco if, allo
stesso livello di indentazione del print,
subito dopo di esso. Allineare l'else
alla riga 8 correttamente con il suo if.
L'ordine corretto e: if esci -> print addio
-> break, else -> elabora comando

Riga 9/10/11 Spostare l'intera definizione della
funzione prima del ciclo while. L'ordine corretto e sempre:
import, definizione delle funzioni,codice principale.

Riga 13: correggere in
datetime.date.today() aggiungendo la classe date mancante.

Riga 16 : rimuovere .time() bastano
datetime.datetime.now() e poi .strftime('%H:%M')
direttamente su quell'oggetto.

Dalla riga 12 Aggiungere prima dell'if: risposta = '' oppure
risposta = 'Comando non riconosciuto.' In questo modo il
return ha sempre un valore valido da restituire, qualunque
cosa accada nei rami condizionali if/elif/else.

EXTRA PAGE NOZIONI

1. Errore di Sintassi (Riga 3) – Il "Blocco Totale"

Descrizione: Un errore sintattico alla riga 3 (es. due punti mancanti dopo while True o parentesi errate).

Conseguenza: SyntaxError. L'interprete Python non avvia nemmeno l'esecuzione. Nessuna riga di codice viene letta, rendendo lo script un semplice file di testo inutilizzabile.

2. Errore Logico (Righe 5-9) – Il "Circuito Interrotto"

Descrizione: Il comando break è indentato allo stesso livello dell'istruzione if anziché al suo interno.

Conseguenza: Errore Semantico. Indipendentemente dall'input dell'utente, il programma esegue il break dopo il primo ciclo. Questo rende la clausola else un "codice morto" (irraggiungibile), causando la chiusura immediata e involontaria dell'assistente.

3. Errore di Struttura/Runtime (Righe 9-11) – "Referenza Anticipata"

Descrizione: Il ciclo while (che chiama la funzione) è posizionato nel file prima della definizione def assistente_virtuale().

Conseguenza: NameError. Python legge il codice dall'alto verso il basso. Quando il ciclo tenta di invocare la funzione, questa non esiste ancora nella memoria del programma, provocando un crash al primo tentativo di utilizzo.

4. Errore di API (Righe 13 e 16) – "Incompatibilità del Modulo"

Descrizione: Utilizzo di metodi inesistenti o chiamate errate al modulo datetime (es. datetime.now() invece di datetime.datetime.now() o l'uso di un metodo inventato).

Conseguenza: AttributeError. Anche se la sintassi fosse corretta, il programma fallisce durante l'esecuzione perché richiede al modulo un'operazione che non appartiene alla sua libreria ufficiale.

Concludendo l’analisi ha identificato cinque errori . Gli errori appartengono a quattro categorie distinte: uno di Sintassi (riga 3), uno Logico (righe 5-9), uno di Struttura (righe 9/11), uno di API (righe 13 e 16) e uno di Variabile (riga 22). Tutti e cinque emergono chiaramente dalla sola lettura del codice.

Conclusioni

Sono stati riscontrati cinque errori . Gli errori appartengono a quattro categorie distinte: uno di Sintassi (riga 3), uno Logico (righe 5-9), uno di Struttura (righe 9/11), uno di API (righe 13 e 16) e uno di Variabile (riga 22). Tutti e cinque emergono chiaramente dalla sola lettura del codice, senza necessita DI eseguirlo.

Leandro Ancona

).